



MEDHEAD

DOCUMENT DE REPORTING

William Burt
01/09/2026

Table des matières

Table des matières	1
1. Introduction	3
1.1 Contexte	3
1.2 Objectifs de la PoC.....	3
1.3 Objectifs du reporting	3
2. Présentation de la solution.....	5
2.1 Architecture globale	5
2.2 Composants et responsabilités	6
2.3 Flux entre les composants	6
3. Choix technologiques	7
3.1 Back-end	7
Java 21 et Spring Boot 4.1.0	7
API REST et documentation OpenAPI.....	7
H2 (base de données).....	7
Autres dépendances notables.....	7
3.2 Front-end	7
3.3 GraphHopper	8
Rôle.....	8
Pourquoi un composant spécialisé et découplé ?.....	8
Intégration avec le backend et découplage du métier.....	8
3.4 Docker.....	8
3.5 CI/CD.....	9
4. Architecture hexagonale légère	10
4.1 Principe	10
4.2 Arborescence.....	11
4.3 Diagramme simplifié de l'architecture hexagonale réelle.....	12
4.4 Valeur ajoutée dans le cadre de la PoC	12
5. Normes, bonnes pratiques et RGPD	14
5.1 Qualité logicielle	14
5.2 RGPD	14
Minimisation et finalités.....	14
Privacy by Design	15
Conservation.....	15
Sécurité.....	15
Synthèse de conformité RGPD	15
6. Tests et validation de la PoC.....	16
6.1 Tests backend	16
Couverture des tests (JaCoco).....	16

6.2 Tests frontend	17
6.3 Tests End-to-End (Cypress).....	17
6.4 Pyramide de tests	17
7. Tests de performance avec Apache JMeter	18
7.1 Configuration de la campagne.....	18
7.2 Résultats	18
7.3 Analyse	19
7.4 Comparaison aux objectifs du projet	19
7.5 Limites des tests de performance	19
7.6 Conclusion et projection en production.....	20
8. Résultats de la PoC	22
8.1 Fonctionnement démontré	22
8.2 Fonctionnement supposé ou non testé	22
9. Difficultés et résolution	23
9.1 Fournisseur de distance externe non concluant sous charge	23
9.2 Clé d'API committée en clair	23
9.3 Stabilisation du pipeline CI multi-services.....	24
10. Enseignements	25
11. Limites et passage en production.....	26
12. Tableau final de conformité	28
13. Conclusion	30

1. Introduction

1.1 Contexte

Le consortium MedHead réunit plusieurs établissements hospitaliers autour d'un objectif commun : Améliorer la qualité de la prise en charge des urgences et la confiance des utilisateurs dans les outils numériques qui la soutiennent. Dans ce cadre, l'équipe d'architecture métier du consortium a formulé l'hypothèse suivante : un système d'intervention d'urgence en temps réel, capable de proposer automatiquement l'hôpital le plus proche disposant à la fois d'un lit libre et de la spécialisation requise, permettrait de réduire le temps de traitement d'une urgence et d'orienter plus fiablement les patients. Ce document présente la PoC que j'ai conçue et développée pour valider cette hypothèse, conformément aux exigences formalisées dans le document « Exigences pour le développement de la PoC » et aux principes d'architecture du consortium. Il décrit les choix d'architecture et d'implémentation, les résultats obtenus, les difficultés rencontrées pendant le développement ainsi que les limites volontairement laissées hors du périmètre de production.

1.2 Objectifs de la PoC

D'après les exigences fournies, la PoC devait démontrer la faisabilité technique d'un service capable de :

- recevoir la spécialité recherchée et la localisation d'un patient ;
- identifier, parmi un référentiel d'hôpitaux, l'établissement le plus proche disposant à la fois de la spécialité demandée et d'au moins un lit disponible ;
- exposer ce résultat via une API RESTful développée en Java, conçue pour pouvoir s'inscrire à terme dans une architecture microservices ;
- être consommée par une interface graphique simple, développée avec un framework JavaScript/TypeScript reconnu ;
- publier un événement métier lors de la réservation d'un lit ;
- garantir un temps de réponse inférieur à 200 ms sous une charge de 800 requêtes par seconde par instance ;
- protéger les données du patient ;
- être validée par une pyramide de tests complète (unitaires, intégration, E2E) incluant des tests de charge ;
- être réutilisable comme brique ou modèle pour les développements futurs, avec une documentation technique et des pipelines CI/CD.

1.3 Objectifs du reporting

Ce document vise à :

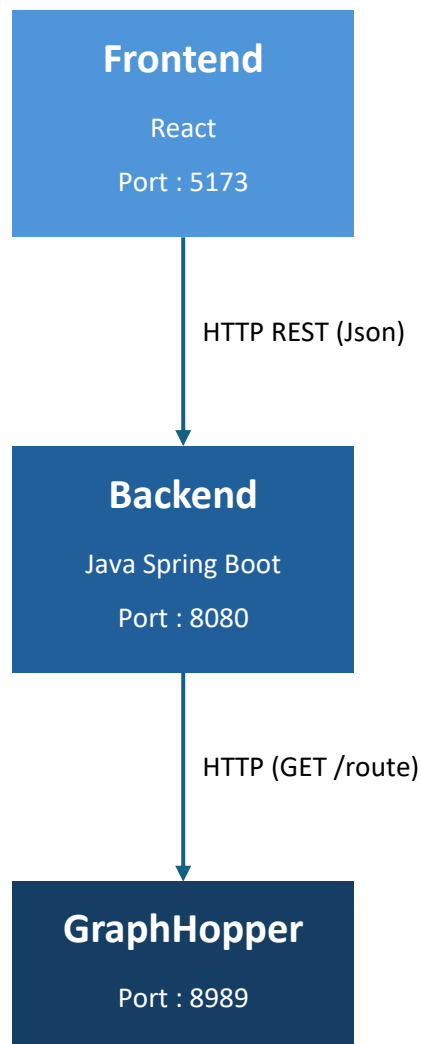
- présenter la solution que j'ai conçue et mise en œuvre et les choix technologiques associés ;

- expliquer comment l'architecture que j'ai mise en œuvre répond aux principes et exigences;
- présenter les choix réalisés au regard des normes, bonnes pratiques et exigences RGPD ;
- présenter et analyser les résultats des tests fonctionnels et des tests de performance ;
- distinguer, pour chaque point, ce qui a été démontré de ce qui reste à faire avant un passage en production ;
- conclure sur la capacité de la PoC à répondre aux objectifs fixés par le consortium.

2. Présentation de la solution

2.1 Architecture globale

J'ai structuré la PoC autour de trois composants indépendants, chacun packagé et déployé séparément. Cette organisation permet de séparer clairement l'interface utilisateur, le traitement métier et le calcul d'itinéraire, tout en conservant une orchestration simple adaptée à une PoC.



2.2 Composants et responsabilités

Composant	Responsabilité	Port
Frontend (React)	Formulaire de recherche d'urgence (spécialité + localisation), affichage du résultat recommandé, liste des hôpitaux, confirmation de réservation de lit.	5173
Backend (Spring Boot)	Expose l'API REST. Contient la règle métier de sélection de l'hôpital et orchestre l'appel au moteur de routage.	8080
GraphHopper	Calcule une distance et un temps de trajet routiers réels entre le patient et chaque hôpital candidat, à partir d'un extrait OpenStreetMap (Pour la PoC, l'Île-de-France).	8989

2.3 Flux entre les composants

Le scénario principal (recommandation d'un hôpital) suit le déroulé suivant :

1. Le frontend envoie GET /recommandations avec la latitude, la longitude du patient et l'identifiant de la spécialité recherchée (RecommandationController).
2. Le backend journalise la demande dans le log d'audit, valide les coordonnées (exceptions si hors bornes GPS), puis délègue au cas d'utilisation RecommenderHopitalUseCase.
3. Le service applicatif RecommenderHopitalService charge la spécialité et la liste des hôpitaux via les ports de sortie, puis appelle le service de domaine EmergencyRoutingService, qui filtre les hôpitaux par spécialité et disponibilité de lits, et sélectionne le plus proche via le port DistanceProviderPort.
4. L'adaptateur DistanceProviderAdapter interroge (via cache mémoire puis, si nécessaire, un appel HTTP) le moteur GraphHopper pour obtenir la distance et la durée réelles du trajet routier.
5. Le résultat (hôpital recommandé, distance en km, durée en minutes) est renvoyé au frontend et journalisé dans le log d'audit.
6. Si l'utilisateur confirme, le frontend appelle POST /reservations : le lit est décrémenté sur l'agrégat Hopital, la nouvelle valeur est persistée, et un événement ReservationLit (avec une référence patient anonymisée générée côté serveur) est publié via EventPublisherPort.

3. Choix technologiques

3.1 Back-end

Java 21 et Spring Boot 4.1.0

Le choix de Java répond directement à l'exigence contractuelle du consortium. Spring Boot fournit l'infrastructure REST (spring-boot-starter-webmvc), la configuration par propriétés externalisées, et l'intégration native avec Actuator pour l'exposition d'un point de santé.

API REST et documentation OpenAPI

Les quatre contrôleurs (RecommandationController, ReservationController, HopitalController, SpecialiteController) exposent une API REST classique (GET/POST, DTOs de sortie dédiés). La dépendance springdoc-openapi-starter-webmvc-ui génère une documentation Swagger/OpenAPI de l'API (:8080/swagger-ui/index.html), répondant à l'exigence de documentation technique.

H2 (base de données)

La base H2 est utilisée en mode mémoire, initialisée à chaque démarrage par schema.sql et data.sql. Ce choix est cohérent avec un usage de PoC : aucune installation de base de données externe n'est nécessaire pour démontrer le comportement fonctionnel. Il constitue en revanche une limite explicite pour une mise en production.

Autres dépendances notables

JUnit 5, Mockito, AssertJ, spring-boot-starter-webmvc-test, spring-boot-resttestclient : socle de test du backend.

3.2 Front-end

Le frontend est développé en React 19 et TypeScript, avec Vite comme outil de build. Le choix de React répond à l'exigence du consortium d'utiliser un framework JavaScript/TypeScript reconnu (Angular, React ou VueJS étaient acceptés).

Pour la réalisation de l'interface frontend, j'ai retenu plusieurs bibliothèques complémentaires afin de couvrir les besoins d'interface utilisateur, de communication avec l'API, de navigation et de validation du comportement applicatif. Ces choix permettent de conserver un frontend simple, maintenable et facilement intégrable au backend de la PoC :

- MUI (Material UI) 9.3.1 : composants d'interface moderne (formulaire de recherche, tableau des hôpitaux).
- Axios : client HTTP centralisé, avec un intercepteur de réponse qui uniformise les erreurs backend en une classe ApiError typée.
- React Router : navigation entre les pages.

- Vitest + React Testing Library : tests unitaires et de composants côté frontend.
- Cypress : tests end-to-end.
- La configuration de l'URL de l'API repose sur la variable d'environnement VITE_API_BASE_URL, injectée au moment du build Docker (--build-arg VITE_API_BASE_URL=...), ce qui permet de pointer vers un backend différent selon l'environnement sans modifier le code. J'ai retenu cette approche pour séparer la configuration du code source, éviter de figer l'infrastructure et limiter le risque de versionner des informations propres à un environnement.

3.3 GraphHopper

GraphHopper est un moteur de calcul d'itinéraires open source, déployé ici comme composant spécialisé et autonome (JAR + extrait OpenStreetMap d'Île-de-France).

Rôle

Il calcule, pour un couple de coordonnées GPS (patient, hôpital), une distance et un temps de trajet routiers réels, qui est une exigence explicite du domaine : « la distance retournée doit toujours être une distance routière réelle simulée, JAMAIS une distance à vol d'oiseau ».

Pourquoi un composant spécialisé et découplé ?

Le calcul d'itinéraire est un problème à forte intensité de calcul et de données. L'isoler dans son propre service évite d'alourdir le backend métier et permet de le faire évoluer, remplacer ou mettre à l'échelle indépendamment.

Intégration avec le backend et découplage du métier

Le domaine ne connaît que l'interface DistanceProviderPort (calculerDistance, calculerTempsTrajet). L'implémentation concrète (DistanceProviderAdapter et GraphHopperClient, avec un cache mémoire RouteCache à durée de vie configurable) est un détail d'infrastructure totalement invisible du domaine et des cas d'utilisation. Je peux démontrer concrètement la valeur de ce découplage : le fournisseur de distance a été remplacé une fois en cours de projet sans modifier ni le domaine ni les use cases.

3.4 Docker

Chacun des trois composants dispose de son propre Dockerfile et d'une image publiée sur GitHub Container Registry.

- **Isolation** : chaque service tourne dans son propre conteneur, avec ses propres dépendances runtime.
- **Reproductibilité** : les Dockerfiles backend et frontend utilisent des builds multi-étapes, garantissant une image identique quel que soit le poste de build.

- **Packaging** : l'image backend embarque uniquement le JAR final; l'image frontend embarque le résultat du build Vite servi par nginx.
- **Déploiement et séparation des composants** : docker-compose.yml orchestre les trois services, avec une variable d'environnement GRAPHHOPPER_API_URL injectée dans le conteneur backend pour pointer vers le conteneur GraphHopper via le réseau Docker interne.

3.5 CI/CD

Le pipeline GitHub Actions est déclenché sur push et pull request vers les branches main et Develop, et enchaîne cinq jobs :

- En parallèle :
 - backend : ./mvnw clean verify (tests unitaires + intégration + JaCoCo), rapport JaCoCo publié comme artefact ;
 - frontend : npm ci, npm run test:run, npm run test:coverage, npm run build — exécuté en parallèle du job backend ;
- cypress : démarre séquentiellement GraphHopper, le backend, puis le frontend (avec attente active sur chaque port via des boucles), exécute npx cypress run, et publie captures d'écran, vidéos et logs applicatifs en cas d'échec ;
- semantic-release : analyse les commits et détermine automatiquement la version, le tag Git et la release GitHub (branche main en version stable, Develop en pré-version « beta ») ;
- docker : construit et publie les images backend et frontend vers ghcr.io, taguées avec la version calculée et « latest ».

4. Architecture hexagonale légère

4.1 Principe

L'architecture hexagonale (ou « ports & adaptateurs », introduite par Alistair Cockburn) place le domaine métier (ici, la logique de sélection de l'hôpital le plus proche disposant d'un lit et de la spécialité requise) au centre de l'application, totalement isolé des détails techniques. Le domaine ne dépend d'aucun framework : dans la PoC, le package domaine ne contient aucun import Spring ni JPA.

Les échanges entre le domaine et le monde extérieur se font exclusivement via des ports, c'est-à-dire des interfaces définies par le domaine lui-même :

- **Ports « entrants » (driving ports)** : les cas d'usage exposés par le domaine (ex. `RecommanderHopitalUseCase`), appelés par les adaptateurs entrants (dans notre cas, les contrôleurs REST).
- **Ports « sortants » (driven ports)** : les besoins exprimés par le domaine vers l'extérieur (ex. `HopitalRepositoryPort`, `CalculDistancePort`, `PublicationEvenementPort`), implémentés par des adaptateurs sortants : l'adaptateur JDBC pour le référentiel des hôpitaux, le client de calcul d'itinéraire, l'adaptateur de journalisation d'événements.

Ce style architectural est particulièrement pertinent pour la PoC : il permet de valider l'hypothèse métier centrale (recommandation d'hôpital) avec un domaine testé en isolation, tout en gardant la possibilité de faire évoluer indépendamment chaque intégration externe (calcul d'itinéraire, publication d'événements, persistance) sans jamais remettre en cause la logique de sélection elle-même. C'est un atout décisif dans un contexte de preuve de concept où le temps de développement doit être investi en priorité sur la démonstration de la valeur métier plutôt que sur l'infrastructure.

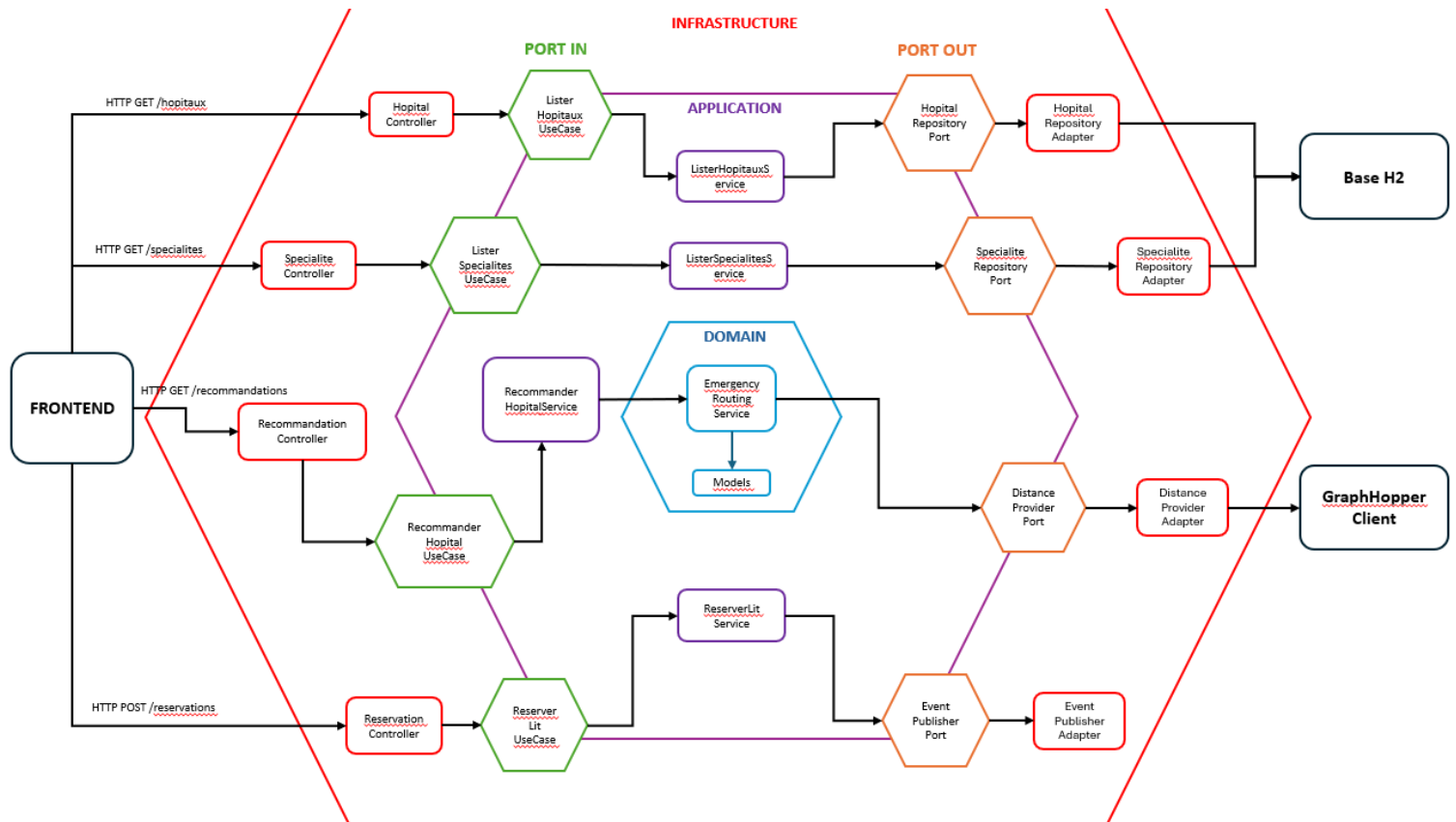
Le terme « légère » est justifié par la simplicité volontaire de certains choix : un seul agrégat métier central (Hopital), pas de bus d'événements réel (`EventPublisherAdapter` journalise plutôt que de publier sur un broker — commentaire explicite dans le code : « en attendant un véritable bus d'événements hors du périmètre de la PoC »), et une persistance JDBC directe plutôt qu'un ORM complet. Cette architecture conserve les bénéfices structurants du découplage (testabilité, remplaçabilité des adaptateurs, indépendance du domaine) sans le coût de mise en place d'une infrastructure événementielle ou d'un ORM complexe, ce qui est cohérent avec le principe C4 du consortium.

4.2 Arborescence

Le backend est donc organisé selon l'arborescence de packages suivante :

Package	Contenu / Rôle
domain/	Cœur métier, aucune dépendance à Spring ni à l'infrastructure
domain/model/	Hopital, Location, Specialite, GroupeSpecialite, ReservationLit
domain/service/	EmergencyRoutingService (règle de sélection de l'hôpital)
domain/port/out/	DistanceProviderPort, EventPublisherPort
domain/exception/	BusinessException et ses sous-classes
application/	Orchestration des cas d'utilisation
application/port/in/	RecommanderHopitalUseCase, ReserverLitUseCase, Lister*UseCase
application/port/out/	HopitalRepositoryPort, SpecialiteRepositoryPort
application/usecase/	RecommanderHopitalService, ReserverLitService, Lister*Service
adapter/in/web/	Adaptateurs entrants (contrôleurs REST + DTOs)
adapter/out/persistence/	Adaptateurs sortants JDBC (H2)
adapter/out/distance/	Adaptateur sortant vers GraphHopper (+ cache)
adapter/out/event/	Adaptateur sortant de publication d'événements (journalisation)
adapter/out/logging/	Journal d'audit métier
config/	DomainConfig, CorsConfig

4.3 Diagramme simplifié de l'architecture hexagonale réelle



4.4 Valeur ajoutée dans le cadre de la PoC

Au-delà de la simplicité de mise en œuvre recherchée, ce style architectural apporte des bénéfices concrets et mesurables tout au long du cycle de vie de la PoC, détaillés ci-dessous :

- **Testabilité** : Le domaine peut être testé unitairement sans base de données, sans serveur HTTP et sans dépendance réseau, en substituant les ports par des doublons de test (mocks/stubs Mockito). Cela permet une couverture de tests rapide et déterministe, conforme au Principe B4 (tests automatisés précoces, complets et appropriés).
- **Évolutivité et remplaçabilité des adaptateurs** : L'adaptateur de calcul de distance peut être remplacé par tout autre fournisseur de cartographie, sans modifier une seule ligne du domaine. De même, l'adaptateur d'événements pourra être remplacé par un véritable bus d'événements (Kafka, par exemple) lors de l'intégration à la plateforme cible du consortium, conformément au Principe B6 (extensibilité pilotée par les événements).
- **Clarté et séparation des préoccupations (Principe B2)** : Chaque composant a une responsabilité unique et ses dépendances sont explicites via les interfaces de port, ce qui limite le couplage fort que le Principe B2 cherche précisément à éviter.

- **Réutilisabilité** : La structuration hexagonale, le contrat d'API REST et le modèle de données peuvent être repris comme gabarit par d'autres microservices du consortium, sans qu'il soit nécessaire de reproduire les choix techniques précis (H2, adaptateur de simulation) propres à la PoC.
- **Alignement avec la conception pilotée par le domaine** : Les invariants métier (par exemple : un Hôpital ne peut être construit sans au moins un GroupeSpecialite et une Specialite associée) sont portés par le domaine lui-même, garantissant qu'aucun état incohérent ne peut être représenté, indépendamment de la couche de persistance retenue.

5. Normes, bonnes pratiques et RGPD

5.1 Qualité logicielle

Principe	Illustration concrète dans le projet
Clean Code / nommage	Le domaine est intégralement en français, cohérent avec le vocabulaire métier du consortium (Hopital, reserverLit(), peutPrendreEnCharge(), disposeDeLits()), un exemple direct de « langage omniprésent » recommandé par le principe C3.
Séparation des responsabilités	Le package domain ne contient aucune annotation Spring, aucune dépendance JDBC ni HTTP. Les DTOs web sont distincts des objets de domaine et la conversion se fait via des méthodes statiques.
Principes SOLID	Responsabilité unique par service applicatif ; inversion de dépendance par les ports ; ouverture à l'extension via de nouvelles implémentations de port (ex. changement de fournisseur de distance) sans modification du code appelant.
Gestion des erreurs	Hiérarchie BusinessException (code métier + statut HTTP porté par l'exception elle-même) et GlobalExceptionHandler centralisant la traduction de toutes les erreurs (métier, validation, panne du fournisseur de distance, erreurs génériques) en réponses HTTP structurées (ErrorResponse), sans jamais exposer de détail technique interne au client.
Validation	Les models valident leurs invariants au constructeur.
Tests	Voir section 6.
Documentation	Javadoc systématique sur les classes ; README couvrant installation, tests, CI/CD, workflow Git et conventions de commit.
Maintenabilité	Découpage hexagonal strict, DTOs distincts du domaine, configuration externalisée par variables d'environnement (GRAPHHOPPER_API_URL, VITE_API_BASE_URL).

5.2 RGPD

La PoC manipule un nombre volontairement restreint de données à caractère potentiellement personnel : la position GPS instantanée du patient (latitude/longitude) et l'identifiant de la spécialité recherchée, transmis à l'endpoint GET /recommandations. Aucun nom, aucune date de naissance, aucun identifiant de dossier médical n'est reçu par le backend à aucun moment du parcours observé dans le code.

Minimisation et finalités

Le contrat d'API ne demande que les données strictement nécessaires à la mise en correspondance patient/hôpital : localisation et spécialité pour la recommandation, identifiant de l'hôpital pour la réservation. Aucune donnée supplémentaire n'est collectée dans les DTOs.

Privacy by Design

Le mécanisme le plus significatif du point de vue RGPD est la classe ReservationLit qui génère elle-même, côté serveur, une référence patient anonymisée ("PAT-" + UUID aléatoire). Il s'agit d'une mesure de Privacy by Design authentique et vérifiable.

Conservation

Les données métier (hôpitaux, lits disponibles) résident en base H2 en mémoire : elles ne survivent pas à un redémarrage de l'application, ce qui limite de fait toute conservation prolongée dans le périmètre de la PoC. Le journal d'audit applicatif est conservé 90 jours par rotation automatique.

Sécurité

Plusieurs mesures concrètes illustrent l'application du principe de sécurité shift-left dès la conception de la PoC (Principe B5) :

- Le point Actuator est explicitement restreint à "health" seul, limitant la surface d'information exposée.
- Le CORS est restreint à l'origine <http://localhost:5173>, avec un commentaire explicite indiquant qu'il doit être restreint à un domaine précis en production.

Synthèse de conformité RGPD

À ce stade, je ne qualifie pas la PoC de « conforme au RGPD » pour un usage en production. J'ai intégré des mécanismes de minimisation et de pseudonymisation dès la conception, mais l'authentification, le chiffrement des échanges et la gestion formalisée des droits restent à mettre en œuvre avant tout traitement de données patient réelles.

6. Tests et validation de la PoC

6.1 Tests backend

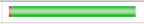
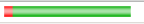
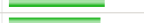
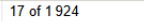
Le backend du projet contient 25 fichiers de test backend (avec 108 méthodes de test

unitaires/intégration), organisés selon la convention Maven (suffixe Test pour les tests unitaires, suffixe IT pour les tests d'intégration) :

- Tests unitaires du domaine : validation des invariants métier (ex. un hôpital sans spécialité ou avec un nombre de lits négatif lève une exception) et de la règle de sélection.
- Tests unitaires des cas d'utilisation : orchestration testée avec les ports simulés (Mockito), sans dépendance à une base de données réelle.
- Tests des adaptateurs entrants : HospitalControllerTest, RecommendationControllerTest, SpecialiteControllerTest, GlobalExceptionHandlerTest.
- Tests des adaptateurs sortants : DistanceProviderAdapterTest, GraphHopperClientTest, DirectionsRequestTest, EventPublisherAdapterTest, HospitalRepositoryAdapterTest, SpecialiteRepositoryAdapterTest.
- Tests d'intégration (Projet11ApplicationIT, HopitalRepositoryAdapterIT, SpecialiteRepositoryAdapterIT) : démarrage d'un contexte applicatif partiel avec la base H2 réelle.

Couverture des tests (JaCoco)

projet11

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.openclassroom.projet11.domain.model		97 %		92 %	5	60	4	86	1	33	0	5
com.openclassroom.projet11		37 %		n/a	1	2	2	3	1	2	0	1
com.openclassroom.projet11.adapter.out.distance		98 %		78 %	3	20	1	69	0	13	0	4
com.openclassroom.projet11.adapter.out.persistence		100 %		100 %	0	17	0	54	0	16	0	3
com.openclassroom.projet11.adapter.in.web.error		100 %		n/a	0	12	0	75	0	12	0	2
com.openclassroom.projet11.adapter.in.web.dto		100 %		n/a	0	12	0	37	0	12	0	6
com.openclassroom.projet11.application.usecase		100 %		n/a	0	9	0	38	0	9	0	4
com.openclassroom.projet11.adapter.in.web		100 %		n/a	0	8	0	33	0	8	0	4
com.openclassroom.projet11.adapter.out.distance.dto		100 %		n/a	0	14	0	24	0	14	0	3
com.openclassroom.projet11.config		100 %		n/a	0	6	0	9	0	6	0	3
com.openclassroom.projet11.domain.service		100 %		n/a	0	5	0	14	0	5	0	1
com.openclassroom.projet11.domain.exception		100 %		n/a	0	7	0	14	0	7	0	5
com.openclassroom.projet11.application.port.in		100 %		n/a	0	3	0	3	0	3	0	3
com.openclassroom.projet11.adapter.out.event		100 %		n/a	0	3	0	5	0	3	0	1
com.openclassroom.projet11.adapter.out.logging		100 %		n/a	0	1	0	1	0	1	0	1
Total	17 of 1 924	99 %	7 of 70	90 %	9	179	7	465	2	144	0	46

La couverture de code (mesurée par JaCoCo) sert d'indicateur complémentaire de la rigueur de la suite de tests : elle quantifie, package par package, la proportion d'instructions et de branches conditionnelles effectivement exercées par les tests unitaires et d'intégration, et permet d'identifier rapidement les zones du code back-end laissées sans preuve de test avant toute mise en production.

Le rapport JaCoCo agrégé (unitaire + intégration) du backend fait état d'une couverture globale de 99 % des instructions et de 90 % des branches. La quasi-totalité des packages de la couche adaptateurs et application atteint 100 % de couverture (persistance, contrôleurs et DTO web, cas d'usage, ports d'entrée,

configuration, services et exceptions du domaine, adaptateurs d'événements et de journalisation), ce qui traduit directement la testabilité apportée par l'architecture hexagonale.

6.2 Tests frontend

Le frontend dispose de tests Vitest + React Testing Library pour la couche API (`client.test.ts`, `hopitaux.test.ts`, `recommandations.test.ts`, `reservations.test.ts`, `specialites.test.ts`) et pour les composants et pages (`HospitalResultCard.test.tsx`, `AppLayout.test.tsx`, `HopitauxPage.test.tsx`, `UrgencesPage.test.tsx`).

Un rapport de couverture v8 fournit les résultats suivants :

Indicateur	Résultat
Statements (instructions)	98,14 % (106 / 108)
Branches	88,37 % (38 / 43)
Functions (fonctions)	100 % (46 / 46)
Lines (lignes)	100 % (101 / 101)

Ces résultats confirment que l'ensemble des lignes et des fonctions du périmètre testé est exécuté par la suite de tests. Les quelques branches non couvertes correspondant principalement à des cas d'erreur secondaires de l'interface.

6.3 Tests End-to-End (Cypress)

Un unique scénario Cypress exécute un parcours complet. Ce test est particulièrement significatif car il reproduit fidèlement le scénario métier du cahier des charges :

- ouverture de la page /urgences ;
- vérification que le référentiel réel de spécialités est chargé ;
- sélection de la spécialité « Cardiologie » et lancement de la recherche ;
- vérification que l'hôpital recommandé n'est jamais l'Hôpital Cochin, qui propose bien la cardiologie mais dispose de 0 lit dans les données de référence (`data.sql`). La règle métier « aucun lit disponible → jamais recommandé » est ainsi vérifiée en conditions réelles, même lorsque l'établissement est géographiquement proche ;
- confirmation de la réservation et vérification de l'affichage d'une référence patient anonymisée (« PAT-... »).

6.4 Pyramide de tests

Le rapport quantitatif ci-dessus (25 fichiers et 108 méthodes de test unitaires/intégration côté backend, une dizaine de fichiers de test côté frontend, un scénario E2E) respecte la forme attendue d'une pyramide de tests : une base large de tests unitaires et d'intégration rapides, et un nombre restreint de tests E2E plus lents et plus coûteux à maintenir, concentrés sur le parcours utilisateur critique.

7. Tests de performance avec Apache JMeter

7.1 Configuration de la campagne

Un plan de test JMeter unique (jmeter/Projet11.jmx) et son rapport agrégé (jmeter/rapport.csv) sont fournis dans le dépôt GitHub.

Paramètre	Valeur (extraite du .jmx)
Endpoint testé	GET /recommandations?longitude=2.3367934&latitude=48.86056&specialiteld=121
Utilisateurs virtuels (threads)	500
Montée en charge (ramp-up)	20 secondes
Durée programmée	120 secondes
Régulation du débit	Constant Throughput Timer, cible 48 000 requêtes/minute, soit 800 requêtes/seconde
Critère de succès	Response Assertion : code de réponse HTTP = 200
Comportement en cas d'erreur	continue (le plan poursuit l'exécution des autres itérations)

La cible de débit choisie (800 requêtes/seconde) correspond très exactement à l'exigence de non-fonctionnel du cahier des charges : « un temps de réponse de moins de 200 millisecondes avec une charge de travail allant jusqu'à 800 requêtes par seconde, par instance de service ».

7.2 Résultats

Indicateur	Résultat
Utilisateurs virtuels	500
Durée réelle observée	≈ 120 s (+ 20 s de montée en charge)
Nombre de requêtes (échantillons)	96 475
Taux d'erreur	0,000 %
Temps de réponse moyen	1 ms
Médiane	1 ms
P90	2 ms
P95	2 ms
P99	20 ms
Minimum	0 ms
Maximum	239 ms

Throughput (débit atteint)	802,42 requêtes/seconde
Débit reçu	343,93 Ko/s
Débit envoyé	146,54 Ko/s

7.3 Analyse

Lors de la campagne que j'ai exécutée un débit de 802,42 requêtes/seconde pendant la durée du test, dépassant légèrement la cible contractuelle de 800 req/s, avec un taux d'erreur nul sur 96 475 requêtes. Le temps de réponse moyen (1 ms) et la médiane (1 ms) sont très largement inférieurs au seuil de 200 ms fixé par le cahier des charges. Le 99^e centile (P99 = 20 ms) confirme que la quasi-totalité des requêtes ont été traitées très en dessous du seuil.

Le temps maximal observé (239 ms) dépasse ponctuellement le seuil de 200 ms. Il s'agit toutefois d'une valeur extrême isolée (un seul échantillon sur 96 475, très éloignée du P99 à 20 ms), cohérente avec un pic ponctuel (par exemple lors de la phase de montée en charge ou d'un accès non mis en cache par RouteCache) plutôt qu'avec une dégradation systémique.

Le cache mémoire RouteCache (adapter/out/distance/RouteCache.java), qui évite un second appel réseau à GraphHopper lorsque la même paire de coordonnées est redemandée dans sa fenêtre de validité, explique vraisemblablement en grande partie la très faible latence moyenne observée : le plan de test utilisant des coordonnées fixes, une large proportion des requêtes a probablement été servie depuis le cache après le tout premier appel.

7.4 Comparaison aux objectifs du projet

Objectif (cahier des charges)	Résultat mesuré	Statut
Temps de réponse < 200 ms	Moyenne 1 ms, médiane 1 ms, P99 20 ms (un maximum isolé à 239 ms)	Validé ; maximum isolé à surveiller
Charge jusqu'à 800 req/s par instance	802,42 req/s soutenues, 0 % d'erreur	Validé

7.5 Limites des tests de performance

Ces résultats doivent toutefois être interprétés avec certaines réserves :

- Un seul endpoint a été testé (GET /recommandations). Les endpoints POST /reservations, GET /hopitaux et GET /specialites n'ont fait l'objet d'aucune campagne de charge fournie.
- Le test a été exécuté en environnement local, non représentatif d'un déploiement réseau réel avec latence et infrastructure de production.

- Les coordonnées géographiques utilisées dans le plan de test sont fixes, ce qui favorise fortement le cache RouteCache et ne représente pas une diversité réelle de requêtes (positions variées sur toute l'Île-de-France).

Les résultats JMeter obtenus constituent une preuve de validation de la PoC pour le scénario et le palier de charge testés ; ils ne garantissent pas, en l'état, les performances en environnement de production réel, notamment pour les autres endpoints et pour une diversité de requêtes plus représentative.

7.6 Conclusion et projection en production

La campagne de tests de performance permet de conclure que les objectifs de performance fixés pour la PoC sont atteints sur le scénario testé.

Ces résultats permettent donc de valider la capacité de l'architecture proposée à répondre aux exigences de performance dans le cadre et les conditions de la PoC. Ils ne permettent cependant pas d'affirmer que ces objectifs seront automatiquement atteints en production. Le test ayant été réalisé en environnement local, sur un seul endpoint et avec des coordonnées géographiques fixes favorisant le fonctionnement du RouteCache, une validation complémentaire est nécessaire avant toute généralisation à un environnement réel.

Les objectifs définis dans le cahier des charges sont néanmoins réalisables en production, à condition que l'architecture et l'infrastructure soient dimensionnées en conséquence. La contrainte de 800 requêtes/seconde doit notamment être considérée comme un objectif par instance de service, et non comme une limite globale de l'application. Une montée en charge horizontale permettrait ainsi d'augmenter la capacité globale en ajoutant plusieurs instances du backend derrière un répartiteur de charge.

Pour garantir durablement ces performances en production, plusieurs mécanismes devraient être mis en œuvre :

- Déploiement de plusieurs instances du backend afin de permettre la montée en charge horizontale et d'éviter qu'une instance unique constitue un point de saturation ;
- Load Balancer pour répartir les requêtes entre les différentes instances ;
- Configuration et dimensionnement adaptés de GraphHopper, notamment en termes de CPU, mémoire et capacité à traiter les requêtes de calcul d'itinéraire ;
- Conservation et dimensionnement du cache RouteCache, avec une politique d'expiration et une stratégie d'invalidation adaptées au volume et à la diversité des requêtes réelles ;
- Monitoring et observabilité, avec notamment le suivi du débit, des temps de réponse par percentile (P50, P95, P99), du taux d'erreur, de la consommation CPU/mémoire et des appels vers GraphHopper ;

- Alerting sur les seuils critiques, par exemple lorsque le P95/P99 dépasse durablement 200 ms ou lorsque le taux d'erreur augmente ;
- Tests de charge réguliers dans un environnement proche de la production, avec des coordonnées variées et des scénarios représentatifs de l'utilisation réelle ;
- Tests de montée en charge et de saturation afin d'identifier la capacité maximale d'une instance et de déterminer le nombre d'instances nécessaires pour absorber les pics de trafic ;
- Tests de résilience, notamment en cas d'indisponibilité ou de ralentissement d'une instance du backend ou de GraphHopper.

Ainsi, la PoC démontre la faisabilité technique de l'objectif de 800 requêtes/seconde avec un temps de réponse inférieur à 200 ms, mais cette démonstration doit être considérée comme une validation de capacité et non comme une garantie de niveau de service en production. La garantie de ces objectifs nécessiterait de compléter la PoC par une infrastructure dimensionnée, une stratégie de montée en charge horizontale, une observabilité complète et des campagnes de tests représentatives des conditions réelles d'exploitation.

8. Résultats de la PoC

8.1 Fonctionnement démontré

Les éléments suivants, chacun rattaché à une preuve de test concrète, démontrent que la PoC répond fonctionnellement aux exigences fixées par le comité d'architecture :

- Recommandation d'un hôpital selon la spécialité et la disponibilité de lits, avec distance et durée de trajet routières réelles (GraphHopper), démontrée par les tests unitaires du domaine, les tests d'intégration, et le test E2E Cypress.
- Exclusion systématique d'un hôpital sans lit disponible, même géographiquement proche, démontrée explicitement par le test Cypress (cas de l'Hôpital Cochin) et par les tests unitaires de `Hopital.peutPrendreEnCharge()`.
- Réservation d'un lit avec décrétement persistant et publication d'un événement contenant une référence patient anonymisée générée côté serveur, démontrée par `ReserverLitServiceTest` et le test E2E.
- Architecture hexagonale opérationnelle et vérifiée par le remplacement effectif d'un adaptateur (fournisseur de distance) sans impact sur le domaine.
- Intégration des trois composants (frontend, backend, GraphHopper) via Docker Compose, avec démarrage automatisé et vérifié dans le pipeline CI.
- Chaîne CI/CD complète et fonctionnelle : tests, couverture, build, E2E, versionnement automatique, publication d'images Docker sur GHCR.
- Tenue de la charge cible (800 req/s) sur l'endpoint de recommandation, avec un temps de réponse quasi systématiquement inférieur à 200 ms.

8.2 Fonctionnement supposé ou non testé

À l'inverse, plusieurs aspects du système restent, à ce stade, supposés fonctionnels sans avoir été formellement validés par un test :

- Comportement sous charge des endpoints `POST /reservations`, `GET /hopitaux` et `GET /specialites` : aucune campagne de charge fournie pour ces routes.
- Comportement en cas de perte de disponibilité du service GraphHopper sous forte charge (le `GlobalExceptionHandler` prévoit une réponse 503 dédiée, mais aucun test de charge combiné à une panne simulée n'a été fourni).
- Comportement multi-instance (plusieurs instances du backend derrière un répartiteur de charge) : la PoC est démontrée avec une seule instance de chaque composant.
- Résilience et reprise après incident (redémarrage, perte de la base H2 en mémoire) : non testée, cohérent avec le caractère volontairement simplifié de la persistance en PoC.

9. Difficultés et résolution

Les difficultés présentées ci-dessous correspondent aux difficultés que j'ai rencontrées au cours du développement. Elles sont retracées dans l'historique Git au travers des commits et des évolutions du code.

9.1 Fournisseur de distance externe non concluant sous charge

Étape	Détail
Problème	Le calcul de distance/durée reposait initialement sur un fournisseur externe, OpenRouteService (API publique par clé d'OpenStreetMap).
Cause	La première campagne de test de charge sur ce fournisseur externe n'a pas donné de résultats satisfaisants. Une dépendance réseau externe, soumise à ses propres limites de débit et à la latence internet, était structurellement mal adaptée à une exigence de moins de 200 ms sous 800 req/s.
Solution	Remplacement de l'adaptateur par un moteur GraphHopper auto-hébergé, embarquant un extrait OpenStreetMap d'Île-de-France, packagé comme composant Docker indépendant. Un cache mémoire (RouteCache) a également été ajouté pour éviter les appels redondants.
Résultat	La campagne JMeter finale démontre un débit de 802,42 req/s avec 0 % d'erreur et une latence moyenne de 1 ms. Objectif de performance atteint après ce changement.
Enseignement	Le découplage apporté par le port DistanceProviderPort a permis ce remplacement complet du fournisseur de distance sans modifier ni le domaine (EmergencyRoutingService) ni les cas d'utilisation (RecommanderHopitalService) : preuve concrète de la valeur de l'architecture hexagonale sur ce projet.

9.2 Clé d'API committée en clair

Étape	Détail
Problème	La clé d'API du fournisseur OpenRouteService a été versionnée dans Git en clair dans src/main/resources/application.properties.
Cause	Absence initiale de règles .gitignore couvrant les fichiers de configuration sensibles (.env) et absence d'externalisation de ce secret dès la première implémentation.
Solution	La clé est remplacée par une variable d'environnement, et le .gitignore est étendu (node_modules, dossiers de build, fichiers .env, scripts de lancement locaux).
Résultat	Aucune clé en clair n'est présente dans la configuration actuelle. Ce risque est par ailleurs devenu sans objet après la migration vers GraphHopper auto-hébergé, qui ne nécessite plus de clé d'API externe.

Enseignement	Une revue de sécurité, même sur une PoC, doit inclure une vérification systématique de l'absence de secrets en clair dans le code versionné dès les premiers commits, conformément au principe consortium B5 (sécurité « shift-left »).
--------------	---

9.3 Stabilisation du pipeline CI multi-services

Étape	Détail
Problème	J'ai mis plusieurs itérations pour stabiliser le job E2E, qui doit démarrer trois processus indépendants (GraphHopper, Spring Boot, Vite) dans un même runner GitHub Actions avant de lancer Cypress.
Cause	Orchestration de services à démarrage asynchrone (le simple lancement d'un processus ne garantit pas sa disponibilité immédiate), lecture d'un fichier OSM volumineux nécessitant Git LFS.
Solution	Le ci.yml actuel démarre chaque service en arrière-plan et attend activement sa disponibilité via des boucles de sondage HTTP avec conditions d'échec explicites (processus arrêté, délai dépassé) et affichage des logs en cas d'échec ; une étape dédiée vérifie Git LFS avant de lancer GraphHopper.
Résultat	Le pipeline actuel enchaîne de façon fiable GraphHopper, Backend, Frontend, Cypress, avec publication automatique des captures d'écran, vidéos et logs applicatifs en cas d'échec, pour faciliter le diagnostic.
Enseignement	L'orchestration de tests E2E multi-services en CI nécessite des vérifications de disponibilité actives (health checks avec délai et journalisation), et non de simples temporisations fixes ; la capture systématique des artefacts de diagnostic (logs, captures, vidéos) est indispensable pour analyser les échecs intermittents en environnement CI.

10. Enseignements

Thème	Enseignement tiré d'une expérience concrète du projet
Architecture hexagonale / découplage	Le remplacement complet du fournisseur de distance (OpenRouteService vers GraphHopper) sans toucher au domaine ni aux cas d'utilisation démontre que l'investissement dans les ports/adapters se rentabilise dès qu'une décision d'infrastructure initiale doit être révisée en cours de projet, y compris sur une PoC de courte durée.
Tests	La pyramide de tests a permis de vérifier explicitement, à trois niveaux (unitaire, intégration, E2E Cypress), la règle métier la plus sensible du projet : ne jamais recommander un établissement sans lit disponible.
GraphHopper	Internaliser un composant à fort besoin de calcul (routage) plutôt que de dépendre d'une API publique tierce a été nécessaire pour tenir un objectif de performance strict (< 200 ms sous 800 req/s) ; cela déplace la complexité vers l'exploitation (image lourde, données OSM) mais sécurise la latence.
CI/CD	L'orchestration d'un environnement E2E multi-services en CI est un sujet à part entière, distinct de l'écriture des tests eux-mêmes ; elle exige des mécanismes explicites d'attente et de diagnostic.
Performances	Un test de charge exécuté tôt a permis de détecter un risque d'architecture (dépendance externe non conforme au SLA) avant qu'il n'atteigne un stade avancé du projet, confirmant l'intérêt du principe consortium B4 (tests automatisés précoces).
RGPD	La pseudonymisation (génération de la référence patient côté serveur) a été conçue directement dans le modèle de domaine plutôt qu'ajoutée après coup, illustrant concrètement le principe de Privacy by Design.
Gestion des environnements	La configuration par variables d'environnement (GRAPHHOPPER_API_URL, VITE_API_BASE_URL) permet de réutiliser les mêmes images Docker dans différents contextes (local, CI, démonstration) sans reconstruction.
Automatisation	Semantic Release, adossé aux Conventional Commits, supprime la gestion manuelle des numéros de version et des tags, réduisant le risque d'erreur humaine lors des livraisons.

11. Limites et passage en production

Les éléments suivants correspondent aux limites que j'ai volontairement conservées dans le périmètre de la PoC.

Domaine	Limite constatée	Évolution nécessaire avant production
Sécurité	Aucune authentification ni autorisation sur les endpoints REST.	Mettre en œuvre une authentification/autorisation. Le principe C3 du consortium recommande OpenID Connect avec les fournisseurs d'identité patients gérés par l'État.
HTTPS	Tous les services communiquent en HTTP simple.	Ajouter la terminaison TLS (reverse proxy / ingress) devant chaque service exposé.
Base de données de production	H2 en mémoire : données perdues à chaque redémarrage, non répliquée.	Migrer vers un SGBD de production. Les adaptateurs actuels reposent sur JdbcTemplate, ce qui facilite le portage vers un moteur SQL standard.
Gestion des secrets	Configuration par variables d'environnement simples (pas de gestionnaire de secrets dédié) ; un secret a par le passé été committé en clair.	Mettre en place un coffre-fort de secrets pour les futurs identifiants (base de données, éventuels services tiers).
Supervision / monitoring	Actuator restreint au seul point « health » ; aucune métrique applicative ni tableau de bord identifiés.	Exposer des métriques techniques et métier et les superviser, conformément au principe consortium B1 (SRE / SLI).
Sauvegarde	Sans objet pour une base en mémoire ; aucune stratégie de sauvegarde définie pour une future base persistante.	Définir une politique de sauvegarde une fois la base de production choisie.
Scalabilité	Une seule instance de chaque service dans docker-compose.yml ; GraphHopper embarque les données OSM dans son image.	Valider la mise à l'échelle horizontale du backend et étudier le partage/la réplication du graphe de routage GraphHopper.
Haute disponibilité	Aucune redondance, aucun mécanisme de bascule entre instances.	Concevoir une architecture redondante conforme au principe consortium B1 (continuité des activités des systèmes critiques pour les patients).
Tests de charge complémentaires	Un seul endpoint et un seul palier de charge testés.	Étendre les campagnes JMeter aux autres endpoints, avec des paliers progressifs et une corrélation aux métriques système.

RGPD	Pas de gestion des droits des personnes concernées.	Réaliser une analyse d'impact formelle et définir les processus d'exercice des droits avant tout traitement de données patient réelles.
------	---	---

12. Tableau final de conformité

Exigence	Réponse apportée	Niveau	Preuve
API RESTful en Java renvoyant le lieu où se rendre	GET /recommandations, implémenté en Java 21 / Spring Boot	Validé	Cf. test E2E Cypress
Technologie Java imposée	Java 21 utilisé sur l'ensemble du backend	Validé	cf. pom.xml du backend
API pouvant s'inscrire à terme dans une architecture microservice	Architecture hexagonale, composant GraphHopper découplé et indépendant, chacun conteneurisé séparément	Validé	Structure de packages, Dockerfiles distincts.
Interface graphique consommant l'API	Frontend React avec formulaire de recherche et tableau des hôpitaux	Validé	frontend/src/pages/UrgencesPage.tsx, HopitauxPage.tsx
Framework JS/TS imposé (Angular/React/VueJS)	React 19 + TypeScript	Validé	frontend/package.json
Protection des données du patient	Minimisation des données transmises, pseudonymisation côté serveur de la référence patient	Validé	Aucune donnée patient en entrée, ni en base
Pyramide de tests (unitaire, intégration, E2E)	25 fichiers de test backend (unitaire + intégration), tests Vitest frontend, 1 scénario Cypress E2E	Validé	src/test/java/..., frontend/src/**/*.test.tsx, cypress/e2e/EmergencyRoutingTest.cy.ts
Tests de stress garantissant la continuité en cas de pic	Campagne JMeter à 800 req/s soutenues, 0 % d'erreur	Validé	jmeter/Projet11.jmx, jmeter/rapport.csv
Code facilement partageable	Dépôt Git public, README détaillé	Validé	README.md, historique Git

Pipelines d'intégration et de livraison continue (CI/CD)	GitHub Actions : backend, frontend, Cypress, semantic-release, Docker/GHCR	Validé	.github/workflows/ci.yml
PoC réutilisable comme brique/modèle pour d'autres modules	Architecture hexagonale documentée, conventions homogènes entre les 4 ressources exposées	Validé	Structure de packages ; README.md
Versionnement Git avec workflow adapté	Branches main/Develop, branches feature/fix/test, Conventional Commits, Semantic Versioning automatique	Validé	Historique Git, .releaserc.json, README section 27
Documentation technique (readme.md)	README couvrant installation, exécution des tests, pipeline, workflow Git détaillé	Validé	README.md (racine, backend, frontend)
Document de reporting (technologies, normes, résultats, enseignements)	Présent document	Validé	

13. Conclusion

La PoC développée répond à l'hypothèse initiale du consortium MedHead sur ses aspects fonctionnels et techniques centraux. Le service recommande de façon fiable, vérifiable par des tests à trois niveaux, l'hôpital le plus proche disposant à la fois de la spécialité recherchée et d'un lit disponible, en s'appuyant sur des distances et durées de trajet routières réelles.

Les choix technologiques (Java/Spring Boot imposé par le consortium, React/TypeScript pour le frontend, GraphHopper comme composant de routage spécialisé) sont justifiés et cohérents avec les contraintes du cahier des charges. L'architecture hexagonale légère mise en œuvre apporte un bénéfice concret : Il a été démontré par le remplacement effectif du fournisseur de calcul de distance en cours de projet, sans impact sur le domaine métier ni sur les cas d'utilisation.

Les résultats de tests, tant fonctionnels (pyramide de tests, scénario E2E reproduisant l'exemple du cahier des charges) que de performance (802 requêtes/seconde soutenues avec un temps de réponse moyen de 1 ms et 0 % d'erreur), constituent des preuves concrètes et vérifiables de la validation de la PoC sur le périmètre testé.

Ces résultats doivent cependant être interprétés au regard des limites de la PoC : un seul endpoint et un seul palier de charge ont été testés, l'authentification et le chiffrement en transit ne sont pas mis en œuvre, la persistance en mémoire n'est pas adaptée à la production, et la supervision applicative ainsi que le traitement formalisé des exigences RGPD relatives aux droits des personnes restent à mettre en place. En production, l'atteinte durable des objectifs de performance dépendra autant de l'application que de l'infrastructure qui l'héberge. Le dimensionnement devra notamment prendre en compte le nombre d'instances du service, les ressources CPU et mémoire, la capacité de GraphHopper, la configuration du réseau et du répartiteur de charge, ainsi que les mécanismes de cache. Une stratégie de montée en charge horizontale pourra être mise en place afin d'absorber les variations de trafic et les pics de charge, tandis qu'une supervision adaptée permettra de suivre les temps de réponse, les taux d'erreur et la consommation des ressources. Enfin, des campagnes de tests de charge réalisées dans un environnement représentatif de la production devront valider le dimensionnement retenu et vérifier le respect des objectifs dans des conditions réalistes.